

Course Outline: Frontend Optimization Strategies

Title: Frontend Optimization Strategies **Learnpack:** + Estrategias de optimización del frontend **Prerequisite:** Midiendo el desempeño del frontend (Core Web Vitals), React fundamentals (props, state, hooks) **Target Audience:** Beginner developers who understand React basics and want to build performant applications **Total Lessons:** 24 **Estimated Total Length:** ~12,500 words **Format:** Mixed (29% hands-on)

Course Description

Performance is not a feature—it's a requirement. Users abandon slow applications, and search engines penalize them. This course teaches you how to transform a working React application into a fast, efficient one.

You already know how to measure performance with Core Web Vitals. Now you'll learn the strategies that actually move those numbers: loading only what's needed, preventing unnecessary re-renders, reusing logic intelligently, and caching data to avoid redundant network requests.

Every optimization technique comes with trade-offs. This course emphasizes not just how to optimize, but when to optimize—and when optimization becomes counterproductive. You'll leave with practical skills and the judgment to apply them correctly.

Course Philosophy

This course emphasizes:

- **Measure before optimizing:** Never guess what's slow—verify with tools
- **Path of least resistance:** Start with the simplest optimization that solves the problem
- **Intentional complexity:** Add optimization only when measurement proves it's needed
- **User perception over metrics:** A fast-feeling app matters more than perfect scores

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Code Loading Strategies	4	~2,200
02 - The Single Responsibility Principle	3	~1,650
03 - Memoization Techniques	4	~2,200
04 - Reusability Patterns	4	~2,200
05 - Caching Strategies	3	~1,650
06 - Assessment	3	~1,700
07 - Conclusion	1	~450
Total	24	~12,500

Section 00: Welcome

00.0 Why Optimization Matters 📖 (~450 words)

Learning Objectives:

- Recognize the business impact of frontend performance
- Understand the relationship between Core Web Vitals and optimization strategies
- Identify the three main areas of frontend optimization

Content Outline:

- The cost of slow: user abandonment, SEO penalties, conversion loss
- Connection to previous learning: you measured FCP, TBT, LCP—now you'll improve them
- The three pillars: load less code, render less often, request less data
- Course roadmap: what you'll learn and build

Transitions: This lesson connects your knowledge of performance metrics to actionable strategies. Section 01 begins with the first pillar: loading less code.

Key Principles:

- Performance is a user experience issue, not a technical curiosity
- Every optimization has a cost—complexity, maintainability, or development time
- The goal is "fast enough," not "theoretically perfect"

Examples:

- A 1-second delay in page load can reduce conversions by 7%
 - Amazon found that every 100ms of latency cost them 1% in sales
 - Google uses page speed as a ranking factor for search results
-

Section 01: Code Loading Strategies

01.0 The Cost of Loading Everything 📖 (~550 words)

Learning Objectives:

- Explain why bundle size affects performance
- Identify the relationship between JavaScript and Total Blocking Time
- Recognize when an application is loading unnecessary code

Content Outline:

- How browsers process JavaScript: download, parse, execute
- The blocking nature of JavaScript execution
- Bundle analysis: understanding what's in your build
- The 244KB rule: keeping initial JavaScript under control
- Real-world scenario: a dashboard that loads chart libraries on the login page

Transitions: Understanding the cost sets up the solution. The next lesson introduces code splitting as the primary technique to reduce initial bundle size.

Key Principles:

- Every byte of JavaScript costs time—twice (download and execution)
- Users pay for code they never use
- The initial load is the most critical moment for user perception

Examples:

- A React app with 5 routes loading all route code upfront
- Chart.js adding 200KB to a bundle when charts appear on one page

- Date-fns vs moment.js: 12KB vs 67KB for the same functionality
-

01.1 Code Splitting Fundamentals 📖 (~550 words)

Learning Objectives:

- Define code splitting and its purpose
- Distinguish between route-based and component-based splitting
- Identify which parts of an application benefit most from splitting

Content Outline:

- What code splitting does: creates multiple smaller bundles
- How bundlers (Webpack, Vite) handle split points
- Route-based splitting: one bundle per page/route
- Component-based splitting: isolating heavy components
- The decision framework: what to split and what to keep together

Transitions: Code splitting creates the bundles. The next lesson shows how lazy loading controls when those bundles are downloaded.

Key Principles:

- Split at natural boundaries: routes, features, or heavy libraries
- Route-based splitting gives the biggest impact with the least effort
- Not everything needs splitting—small components add overhead without benefit

Examples:

- An e-commerce site: split checkout flow from browsing flow
 - A dashboard: split admin panels from user views
 - A blog: keep article rendering together, split comment system
-

01.2 Lazy Loading in React 📖 (~550 words)

Learning Objectives:

- Implement lazy loading using React.lazy and Suspense
- Configure appropriate fallback UI during loading
- Handle loading errors gracefully

Content Outline:

- React.lazy(): converting static imports to dynamic imports
- Suspense: declaring loading boundaries
- Fallback strategies: spinners, skeletons, or content placeholders
- Error boundaries: catching failed imports
- Next.js dynamic imports: the framework approach

Transitions: You understand the concepts. The next lesson puts them into practice with route-based splitting—the highest-impact optimization.

Key Principles:

- Lazy loading defers the cost, it doesn't eliminate it
- Good fallbacks improve perceived performance even when loading
- Always plan for failure—networks are unreliable

Examples:

- `const Chart = lazy(() => import('./Chart'))` with a skeleton fallback
 - Route-level splitting with Suspense boundaries per route
 - Error boundary showing "Failed to load, click to retry"
-

01.3 Implement Route-Based Splitting 📖 (~550 words)

Challenge Prompt: You have a React application with three routes: Home, Dashboard, and Settings. Currently, all route components are imported statically, meaning the entire application loads even when the user only visits the Home page.

Your task: Convert the static imports to lazy imports with appropriate Suspense boundaries. Each route should only load when the user navigates to it.

Requirements:

- Use `React.lazy()` for each route component
- Wrap routes with `Suspense` and provide a loading fallback
- The fallback should display "Loading..." text centered on the screen

Solution Code:

```
import { lazy, Suspense } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

const Home = lazy(() => import('./pages/Home'));
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Settings = lazy(() => import('./pages/Settings'));

function LoadingFallback() {
  return (
    <div style={{ display: 'flex', justifyContent: 'center', alignItems: 'center', height: '100vh' }}>
      Loading...
    </div>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<LoadingFallback />}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/settings" element={<Settings />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Challenge Code:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import Dashboard from './pages/Dashboard';
import Settings from './pages/Settings';

// TODO: Convert static imports above to lazy imports
// TODO: Create a LoadingFallback component that centers "Loading..." text
```

```
export default function App() {
  return (
    <BrowserRouter>
      {/* TODO: Wrap Routes with Suspense and provide fallback */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Section 02: The Single Responsibility Principle

02.0 SRP for Frontend Components 📖 (~550 words)

Learning Objectives:

- Define the Single Responsibility Principle in the context of React components
- Identify signs that a component has too many responsibilities
- Explain how SRP improves both performance and maintainability

Content Outline:

- SRP defined: a component should have one reason to change
- The connection to performance: smaller components = smaller re-render scope
- Signs of violation: components with many props, multiple state variables, mixed concerns
- The "and" test: if you describe a component with "and," it might need splitting
- SRP is about cohesion, not size

Transitions: Knowing what SRP means is the first step. The next lesson shows practical techniques for separating concerns in existing components.

Key Principles:

- A component's responsibility is defined by what changes it
- Splitting components improves testability, reusability, and performance
- Over-splitting creates its own problems—find the natural boundaries

Examples:

- A UserCard that fetches data, formats dates, handles clicks, and renders UI (4 responsibilities)
- A Modal component that should only render—not manage its own open/close state
- A Form component: validation logic vs. rendering logic vs. submission logic

02.1 Separating Concerns in Practice 📖 (~550 words)

Learning Objectives:

- Apply common separation patterns: container/presentational, hooks extraction
- Recognize the boundary between UI logic and business logic
- Decide when separation adds value versus unnecessary complexity

Content Outline:

- Container/Presentational pattern: data-fetching wrappers + pure rendering

- Extracting logic to custom hooks: when state management becomes reusable
- The rule of three: extract on the third duplication, not the first
- When NOT to separate: premature abstraction costs more than it saves
- Practical refactoring workflow: identify, extract, verify

Transitions: You understand the principles. The next challenge applies them to a real component that violates SRP.

Key Principles:

- Separation should make code easier to understand, not harder
- Business logic belongs in hooks or utilities, not in JSX
- Extract when you have a clear reason, not "just in case"

Examples:

- A ProductList extracting `useProducts()` hook for data fetching
- A PriceDisplay separating formatting logic into a utility function
- A SearchBar keeping local input state but lifting search execution to parent

02.2 Refactor a Monolithic Component 📖 (~550 words)

Challenge Prompt: The `UserProfile` component below violates SRP. It fetches data, formats dates, handles loading states, and renders the UI—all in one place.

Your task: Extract the data fetching and loading logic into a custom hook called `useUser`. The component should only handle rendering.

Requirements:

- Create `useUser(userId)` hook that returns `{ user, isLoading, error }`
- The hook handles the fetch and state management
- The component only receives data and renders

Solution Code:

```
import { useState, useEffect } from 'react';

function useUser(userId) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setIsLoading(true);
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => {
        setUser(data);
        setIsLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setIsLoading(false);
      });
  }, [userId]);

  return { user, isLoading, error };
}
```

```
export default function UserProfile({ userId }) {
  const { user, isLoading, error } = useUser(userId);

  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
}
```

Challenge Code:

```
import { useState, useEffect } from 'react';

// TODO: Create useUser(userId) hook that returns { user, isLoading, error }

export default function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setIsLoading(true);
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => {
        setUser(data);
        setIsLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setIsLoading(false);
      });
  }, [userId]);

  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
}
```

Section 03: Memoization Techniques

03.0 Understanding Re-renders 📖 (~550 words)

Learning Objectives:

- Explain what triggers a React re-render

- Trace re-render propagation through a component tree
- Identify unnecessary re-renders using React DevTools

Content Outline:

- The re-render trigger: state changes, prop changes, parent re-renders
- The cascade effect: parent renders cause all children to render
- Reference equality: why `{}` `!==` `{}` matters for props
- Using React DevTools Profiler to visualize re-renders
- The cost of re-renders: usually cheap, sometimes expensive

Transitions: Understanding when re-renders happen reveals when they're wasteful. The next lesson introduces `useMemo` to prevent expensive recalculations during re-renders.

Key Principles:

- Re-renders are not inherently bad—React is optimized for them
- The problem is expensive work inside components that re-render often
- Measure before assuming a re-render is problematic

Examples:

- A list of 1000 items re-rendering when unrelated state changes
 - A filter function running on every keystroke in a search input
 - A chart component recalculating data on every parent update
-

03.1 `useMemo` for Expensive Calculations 📖 (~550 words)

Learning Objectives:

- Implement `useMemo` to cache computation results
- Define appropriate dependency arrays for memoization
- Distinguish between calculations worth memoizing and those that aren't

Content Outline:

- `useMemo` syntax: `useMemo(() => computation, [dependencies])`
- What "expensive" means: filtering large arrays, complex calculations, data transformations
- The dependency array: when to recalculate
- The overhead of `useMemo`: memory cost and comparison cost
- The rule: if you're not sure you need it, you don't

Transitions: `useMemo` caches values. The next lesson covers `React.memo`, which caches entire component renders.

Key Principles:

- `useMemo` is for expensive calculations, not all calculations
- Incorrect dependencies cause bugs that are hard to trace
- Premature memoization adds complexity without benefit

Examples:

- Memoizing a filtered/sorted list of 500+ items
 - NOT memoizing `const fullName = first + ' ' + last`
 - Memoizing a computed value derived from multiple state variables
-

03.2 `React.memo` for Component Optimization 📖 (~550 words)

Learning Objectives:

- Apply `React.memo` to prevent unnecessary component re-renders
- Configure custom comparison functions when needed
- Recognize when `React.memo` provides value versus overhead

Content Outline:

- `React.memo`: wrapping components to skip re-renders when props haven't changed
- Shallow comparison: how `React.memo` checks prop equality
- The object/function trap: new references break memoization
- `useCallback`: stabilizing function props for memoized children
- When `React.memo` helps: expensive renders, frequent parent updates, stable props

Transitions: You now have two memoization tools. The next challenge combines them to optimize a component with both expensive calculations and frequent re-renders.

Key Principles:

- `React.memo` only helps when props actually stay the same
- Passing new objects or functions on every render defeats memoization
- Profile before and after to verify improvement

Examples:

- A `<ExpensiveChart data={data} />` wrapped in `React.memo`
- Using `useCallback` for `onClick` handlers passed to memoized children
- A component receiving primitive props (strings, numbers) benefits most

03.3 Apply Memoization Correctly 🖥️ (~550 words)

Challenge Prompt: The `ProductList` component receives a large array of products and a search term. On every render, it filters the products—even when neither the products nor the search term has changed.

Your task: Use `useMemo` to cache the filtered results. Only recalculate when `products` or `searchTerm` actually change.

Requirements:

- Memoize the `filteredProducts` computation
- Ensure the dependency array is correct
- Do not modify the rendering logic

Solution Code:

```
import { useMemo } from 'react';

export default function ProductList({ products, searchTerm }) {
  const filteredProducts = useMemo(() => {
    return products.filter(product =>
      product.name.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }, [products, searchTerm]);

  return (
    <ul>
      {filteredProducts.map(product => (
        <li key={product.id}>{product.name} - ${product.price}</li>
      ))}
    </ul>
  );
}
```

Challenge Code:

```
export default function ProductList({ products, searchTerm }) {
  // TODO: Memoize this filtering operation
  const filteredProducts = products.filter(product =>
    product.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  return (
    <ul>
      {filteredProducts.map(product => (
        <li key={product.id}>{product.name} - ${product.price}</li>
      ))}
    </ul>
  );
}
```

Section 04: Reusability Patterns

04.0 Custom Hooks: The Modern Approach 📖 (~550 words)

Learning Objectives:

- Create custom hooks to extract and share stateful logic
- Apply naming conventions and rules of hooks
- Identify logic that belongs in a hook versus a utility function

Content Outline:

- Custom hooks: functions that use other hooks
- The "use" prefix: convention and tooling requirement
- What goes in a hook: state, effects, context—anything using React features
- What stays as a utility: pure functions without React dependencies
- Composition: hooks calling other hooks

Transitions: Custom hooks are the modern standard. The next lesson covers older patterns you'll encounter in existing codebases.

Key Principles:

- Custom hooks are just functions—they follow normal JavaScript composition
- Extract hooks when logic is reused OR when it clarifies component responsibility
- Hooks don't add to the component tree—they're invisible to React DevTools

Examples:

- `useLocalStorage(key, initialValue)` for persisted state
- `useDebounce(value, delay)` for search input optimization
- `useWindowSize()` for responsive behavior

04.1 HOCs and Render Props: Legacy Patterns 📖 (~550 words)

Learning Objectives:

- Recognize Higher-Order Components (HOCs) in existing code
- Understand the render props pattern and its use cases
- Compare these patterns to custom hooks for the same problems

Content Outline:

- HOCs: functions that take a component and return an enhanced component
- The wrapper problem: HOCs add layers to the component tree
- Render props: passing render functions as props
- Why hooks replaced these: simpler mental model, no wrapper hell
- When you'll still see them: libraries (withRouter, connect), legacy code

Transitions: Understanding legacy patterns helps you work with existing code. The next lesson addresses when optimization itself becomes a problem.

Key Principles:

- HOCs and render props solved real problems—hooks just solve them better
- You don't need to use these patterns, but you need to read them
- Converting HOCs to hooks is often a valuable refactor

Examples:

- `withAuth(Component)` HOC checking authentication before render
 - A `<DataFetcher render={data => <Display data={data} />} />` render prop
 - The same logic as a `useAuth()` hook—cleaner, more flexible
-

04.2 When Optimization Becomes a Problem 📖 (~550 words)

Learning Objectives:

- Identify premature optimization anti-patterns
- Recognize the costs of over-optimization
- Apply a measurement-first approach to performance work

Content Outline:

- Micro-splitting: lazy loading tiny components adds more overhead than it saves
- Memoization everywhere: comparison costs exceed saved render costs
- Wrapper hell: five HOCs deep makes debugging impossible
- The 80/20 rule: 80% of performance gains come from 20% of optimizations
- The decision framework: measure, identify the bottleneck, fix only that

Transitions: Knowing what not to do is as important as knowing what to do. The next challenge asks you to apply judgment about when to optimize.

Key Principles:

- Optimization without measurement is guessing
- Code complexity is a real cost—factor it into decisions
- "Premature optimization is the root of all evil" —Donald Knuth

Examples:

- Lazy loading a 2KB icon component (overhead exceeds benefit)
 - `useMemo` on `array.length` (comparison costs more than the "computation")
 - Wrapping every component in `React.memo` "just in case"
-

04.3 Extract Logic into a Custom Hook 📖 (~550 words)

Challenge Prompt: Two components, `SearchResults` and `AutocompleteDropdown`, both implement the same debounced search logic. This violates DRY and makes maintenance harder.

Your task: Create a `useDebounce` hook that both components can use. The hook should delay updating a value until the user stops typing for a specified time.

Requirements:

- `useDebounce(value, delay)` returns the debounced value
- The debounced value updates only after `delay` milliseconds of no changes
- Clean up the timeout on unmount or when value changes

Solution Code:

```
import { useState, useEffect } from 'react';

export function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

// Usage in component:
// const debouncedSearch = useDebounce(searchTerm, 300);
```

Challenge Code:

```
import { useState, useEffect } from 'react';

// TODO: Create useDebounce(value, delay) hook
// - Return a debounced version of the value
// - Update the debounced value after `delay` ms of no changes
// - Clean up timeout on unmount or when value changes

export function useDebounce(value, delay) {
  // Your code here
}
```

Section 05: Caching Strategies

05.0 The Fastest Request Is No Request 📖 (~550 words)

Learning Objectives:

- Explain why client-side caching improves performance
- Identify data that benefits from caching versus data that must be fresh
- Recognize the trade-off between freshness and speed

Content Outline:

- Network latency: even fast servers are slower than local memory
- What to cache: user data, configuration, static resources, computed results

- What not to cache: real-time data, security-sensitive information
- Cache invalidation: the "two hard problems" in computer science
- Cache layers: browser, CDN, application state, component state

Transitions: Understanding why we cache sets up the how. The next lesson covers the stale-while-revalidate pattern—a practical caching strategy.

Key Principles:

- Caching trades freshness for speed—know when that trade-off is acceptable
- Stale data that appears instantly often beats fresh data that takes 2 seconds
- Clear cache invalidation strategy prevents showing wrong data

Examples:

- User profile data: cache for the session, invalidate on logout
 - Product prices: don't cache (users expect current prices)
 - Navigation menu items: cache aggressively (rarely changes)
-

05.1 Stale-While-Revalidate Pattern 📖 (~550 words)

Learning Objectives:

- Implement the stale-while-revalidate pattern manually
- Understand how libraries like TanStack Query and SWR apply this pattern
- Configure appropriate cache times for different data types

Content Outline:

- The SWR flow: show cached data immediately, fetch fresh data in background, update when ready
- Manual implementation: state for data, state for loading, effect for fetch
- Library approach: TanStack Query's `staleTime` and `cacheTime`
- Next.js approach: fetch caching and revalidation
- Choosing cache duration: user expectations vs. data volatility

Transitions: You understand the pattern conceptually. The next challenge implements a basic version to solidify the concept.

Key Principles:

- Show something fast, then improve it—users prefer responsive UIs
- Background revalidation is invisible to users when done well
- Libraries handle edge cases (race conditions, deduplication) you'd otherwise miss

Examples:

- A dashboard showing yesterday's data instantly while fetching today's
 - A user list that refreshes every 5 minutes in the background
 - A product page that revalidates on window focus
-

05.2 Implement Basic Caching 📖 (~550 words)

Challenge Prompt: The `UserList` component fetches users every time it mounts—even if the data was fetched seconds ago. This creates unnecessary network requests.

Your task: Implement a simple cache using a module-level variable. If cached data exists and is less than 60 seconds old, return it immediately. Otherwise, fetch fresh data.

Requirements:

- Cache the fetched users with a timestamp
- Return cached data if it's less than 60 seconds old
- Fetch and cache new data if cache is stale or empty

Solution Code:

```
import { useState, useEffect } from 'react';

let cache = { data: null, timestamp: 0 };
const CACHE_DURATION = 60000; // 60 seconds

export default function UserList() {
  const [users, setUsers] = useState(cache.data || []);
  const [isLoading, setIsLoading] = useState(!cache.data);

  useEffect(() => {
    const now = Date.now();
    const isCacheValid = cache.data && (now - cache.timestamp) < CACHE_DURATION;

    if (isCacheValid) {
      setUsers(cache.data);
      setIsLoading(false);
      return;
    }

    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        cache = { data, timestamp: Date.now() };
        setUsers(data);
        setIsLoading(false);
      });
  }, []);

  if (isLoading) return <div>Loading...</div>;

  return (
    <ul>
      {users.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

Challenge Code:

```
import { useState, useEffect } from 'react';

// TODO: Create cache object with data and timestamp
// TODO: Define CACHE_DURATION (60 seconds in milliseconds)

export default function UserList() {
  const [users, setUsers] = useState([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    // TODO: Check if cache is valid (exists and less than 60 seconds old)
    // TODO: If valid, use cached data
    // TODO: If not valid, fetch and update cache
  });
}
```

```

    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setIsLoading(false);
      });
  }, []);

  if (isLoading) return <div>Loading...</div>;

  return (
    <ul>
      {users.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}

```

Section 06: Assessment

06.0 Optimization Decision Making 📖 (~550 words)

Learning Objectives:

- Apply the optimization decision framework to real scenarios
- Weigh trade-offs between different optimization strategies
- Justify optimization choices with measurement-based reasoning

Content Outline:

- Review: the three pillars (load less, render less, request less)
- Decision tree: how to choose the right optimization
- Case study: optimizing a slow dashboard
- The audit checklist: what to look for in performance reviews
- Preparing for the assessment: key concepts to remember

Transitions: This lesson synthesizes everything you've learned. The next challenge tests your ability to apply multiple optimizations to a single component.

Key Principles:

- Start with the biggest bottleneck, not the easiest fix
- Multiple small optimizations compound into significant improvements
- Document why you optimized—future you will thank present you

Examples:

- Dashboard audit: large bundle (split), slow list (virtualize), redundant fetches (cache)
- Prioritizing: 3-second initial load vs. 200ms re-render delay
- When to stop: diminishing returns below perceptible thresholds

06.1 Optimize a React Application 📖 (~600 words)

Challenge Prompt: The `Dashboard` component has three performance problems:

1. It imports a heavy `Chart` component that's only visible when `showChart` is true
2. It recalculates `expensiveData` on every render, even when `items` hasn't changed

3. It creates a new `handleClick` function on every render, breaking memoization of `<ActionButton />`

Your task: Apply the correct optimization for each problem.

Requirements:

- Lazy load the Chart component
- Memoize the expensive calculation
- Stabilize the click handler with `useCallback`

Solution Code:

```
import { useState, useMemo, useCallback, lazy, Suspense } from 'react';
import ActionButton from './ActionButton';

const Chart = lazy(() => import('./Chart'));

export default function Dashboard({ items }) {
  const [showChart, setShowChart] = useState(false);

  const expensiveData = useMemo(() => {
    return items.reduce((acc, item) => {
      return acc + complexCalculation(item);
    }, 0);
  }, [items]);

  const handleClick = useCallback(() => {
    console.log('Action performed');
  }, []);

  return (
    <div>
      <h1>Dashboard</h1>
      <p>Computed Value: {expensiveData}</p>
      <ActionButton onClick={handleClick} />
      <button onClick={() => setShowChart(!showChart)}>Toggle Chart</button>
      {showChart && (
        <Suspense fallback=<div>Loading chart...</div>>
        <Chart data={items} />
      </Suspense>
      )}
    </div>
  );
}

function complexCalculation(item) {
  return item.values.reduce((a, b) => a + b, 0) * item.multiplier;
}
```

Challenge Code:

```
import { useState } from 'react';
import Chart from './Chart';
import ActionButton from './ActionButton';

export default function Dashboard({ items }) {
  const [showChart, setShowChart] = useState(false);

  // TODO: Memoize this calculation
```



```

const expensiveData = items.reduce((acc, item) => {
  return acc + complexCalculation(item);
}, 0);

// TODO: Stabilize this function reference
const handleClick = () => {
  console.log('Action performed');
};

return (
  <div>
    <h1>Dashboard</h1>
    <p>Computed Value: {expensiveData}</p>
    <ActionButton onClick={handleClick} />
    <button onClick={() => setShowChart(!showChart)}>Toggle Chart</button>
    {/* TODO: Lazy load Chart component */}
    {showChart && <Chart data={items} />}
  </div>
);
}

function complexCalculation(item) {
  return item.values.reduce((a, b) => a + b, 0) * item.multiplier;
}

```

06.2 Frontend Optimization Quiz 🧠 (~550 words)

Learning Objectives:

- Demonstrate understanding of code splitting and lazy loading
- Identify appropriate use cases for memoization
- Apply optimization decision-making to realistic scenarios

Question 1: A React application loads slowly on initial page load. The bundle analyzer shows that a 150KB charting library is included, but charts only appear on the `/analytics` route. What optimization should you apply first?

- A) Wrap all components in `React.memo` B) Add `useMemo` to all computed values C) Lazy load the charting library with `React.lazy`
 D) Implement service worker caching

Correct Answer: C

Explanation: The charting library is loading on every page but only used on one route. Lazy loading with `React.lazy` ensures it only downloads when the user visits `/analytics`, reducing initial bundle size by 150KB.

Question 2: You have a component with this code: `const sum = useMemo(() => a + b, [a, b])`. Is this a good use of `useMemo`?

- A) Yes, all calculations should be memoized B) No, the comparison overhead exceeds the computation cost C) Yes, but only if `a` and `b` are objects D) No, `useMemo` only works with arrays

Correct Answer: B

Explanation: Adding two numbers is an extremely fast operation. The overhead of storing the result, maintaining dependencies, and running comparisons on every render costs more than just performing the addition.

Question 3: A parent component re-renders frequently. Its child component is wrapped in `React.memo`, but still re-renders every time. The child receives an `onClick` prop. What's likely causing the re-renders?

A) React.memo doesn't work with event handlers B) The parent is creating a new function reference on each render C) The child has internal state changes D) React.memo only prevents renders from state changes, not prop changes

Correct Answer: B

Explanation: React.memo uses shallow comparison. If the parent defines `onClick={() => doSomething()}` inline, a new function is created on every render. React.memo sees different references and re-renders the child. Use `useCallback` to stabilize the function reference.

Question 4: You're implementing caching for an API that returns user profile data. The data rarely changes but must be accurate when the user updates their profile. What caching strategy is most appropriate?

A) Never cache—always fetch fresh data B) Cache indefinitely with no invalidation C) Stale-while-revalidate with cache invalidation on profile updates D) Cache for 24 hours regardless of updates

Correct Answer: C

Explanation: Stale-while-revalidate shows cached data immediately for fast perceived performance while fetching fresh data in the background. Invalidating the cache when the user updates their profile ensures accuracy when it matters most.

Question 5: Which statement about custom hooks versus Higher-Order Components (HOCs) is correct?

A) HOCs provide better performance than custom hooks B) Custom hooks add wrapper components to the React tree C) Custom hooks are the modern replacement for logic-sharing patterns D) HOCs should be used for all new React code

Correct Answer: C

Explanation: Custom hooks solve the same logic-sharing problems as HOCs but without adding wrapper components to the tree, avoiding "wrapper hell." They're simpler to compose, test, and debug. HOCs remain in legacy code and some libraries, but custom hooks are the standard for new development.

Evaluation Criteria:

- 5 correct: Excellent understanding of frontend optimization
 - 4 correct: Good grasp with minor gaps
 - 3 correct: Foundational understanding, review weak areas
 - 2 or fewer: Revisit course material before proceeding
-

Section 07: Conclusion

07.0 The Optimization Mindset (~450 words)

Content Outline:

What You Accomplished: You now have a complete toolkit for frontend optimization. You can reduce initial load times with code splitting and lazy loading. You can prevent wasteful re-renders with `useMemo` and `React.memo`. You can share logic cleanly with custom hooks. And you can eliminate redundant network requests with caching strategies.

More importantly, you've learned when NOT to optimize. You understand that premature optimization creates complexity without benefit, and that measurement must precede action.

Connection to the Bigger Picture: Frontend optimization is one piece of application performance. The backend work you'll learn next—Python, APIs, databases—affects the data your frontend receives. Full-stack performance thinking considers both sides: fast API responses AND efficient rendering of those responses.

The patterns here (SRP, caching, lazy loading) appear throughout software development. Separating concerns improves any codebase. Caching speeds up any system. Loading only what's needed is universally good practice.

Next Steps:

- Profile a real application with React DevTools Profiler
- Implement lazy loading in your current project
- Read the TanStack Query or SWR documentation for production-grade caching
- Explore Next.js built-in optimizations: Image component, font optimization, static generation

Resources for Continued Learning:

- React documentation on Performance
- web.dev Core Web Vitals guide
- TanStack Query documentation
- Chrome DevTools Performance panel tutorial

Final Thought: The best optimization is often deleting code. Fewer features mean smaller bundles. Simpler components mean fewer re-renders. Less data means faster networks. Before adding optimization complexity, ask: can I simplify instead?

Performance work is never finished—but it doesn't have to be perfect. A fast-enough application that ships beats a theoretically-optimal one that doesn't. Measure, fix the biggest problem, ship, repeat.

You're ready to build applications that users enjoy using—because they don't have to wait.
